HOMELAB

1. GIỚI THIỆU

1.1. Bối cảnh và vấn đề nghiên cứu

Trong bối cảnh xã hội hiện đại năm 2026, với sự phát triển nhanh chóng của công nghệ số và trí tuệ nhân tạo (AI), nhu cầu chăm sóc sức khỏe cá nhân hóa ngày càng tăng cao. Đại dịch COVID-19 đã thúc đẩy sự chuyển dịch từ dịch vụ y tế truyền thống sang các mô hình tiện lợi hơn, như xét nghiệm và tư vấn sức khỏe tại nhà. Tuy nhiên, các ứng dụng hiện có trên thị trường (như Y - HOMEMEDI, Kantesti, Biolabs Diagnostic) thường yêu cầu người dùng tương tác qua giao diện phức tạp với nhiều nút bấm, dẫn đến trải nghiệm không thân thiện, đặc biệt với người cao tuổi hoặc người bận rộn.

Vấn đề chính là thiếu một hệ thống trợ lý AI thông minh, cho phép người dùng đặt lịch xét nghiệm và nhận tư vấn sức khỏe qua ngôn ngữ tự nhiên (natural language), giống như trò chuyện với trợ lý ảo.

→ Xây dựng một ứng dụng web kiểu AI Assistant để giải quyết vấn đề này, tập trung vào dịch vụ lấy mẫu xét nghiệm tại nhà và tư vấn cơ bản dựa trên dữ liệu đầu vào.

1.2. Mục tiêu khóa luận

- **Mục tiêu chính:** Xây dựng một ứng dụng web "HomeLab" sử dụng AI để hỗ trợ đặt lịch xét nghiệm tại nhà và tư vấn sức khỏe cơ bản qua giao diện chat đơn giản.
- **Mục tiêu cụ thể:**
  - Phát triển hệ thống phân tích ngôn ngữ tự nhiên để xử lý yêu cầu người dùng (ví dụ: "Tôi muốn đặt lịch xét nghiệm máu vào 10h ngày 15/1").
  - Tích hợp cơ sở dữ liệu để quản lý lịch hẹn và lịch sử tư vấn.
  - Đảm bảo ứng dụng dễ sử dụng, giao diện đơn giản, an toàn dữ liệu và phù hợp với xu hướng AI y tế năm 2026.
  - Đánh giá hiệu suất hệ thống qua thử nghiệm và demo.

1.3. Phạm vi và giới hạn

- **Phạm vi:**

    - Tập trung vào dịch vụ tư vấn / đặt lịch xét nghiệm lấy mẫu tại nhà (máu, nước tiểu, v.v.) và tư vấn sức khỏe cơ bản (dựa trên triệu chứng nhập vào).
    - Giao diện chính là chat-based
    - Hỗ trợ ngôn ngữ tiếng Việt, với người dùng cá nhân ở Việt Nam (có thể mở rộng hỗ trợ cả Tiếng Anh)
- **Giới hạn:**
    - Không thực hiện chẩn đoán y tế thực tế (chỉ tư vấn cơ bản và đưa ra lời khuyên, khuyến nghị liên hệ bác sĩ).
    - Không tích hợp thanh toán hoặc kết nối với phòng lab thực
    - AI sử dụng model pre-trained, không huấn luyện từ đầu để tránh phức tạp.

2. TỔNG QUAN VỀ ỨNG DỤNG

2.1. Mô tả tổng quát

"HomeLab" là một ứng dụng web kiểu AI Assistant, cho phép người dùng tương tác qua chat để đặt lịch xét nghiệm tại nhà và nhận tư vấn sức khỏe. Thay vì UI đầy đủ như bình thường với form và nút bấm, người dùng chỉ cần gõ yêu cầu tự nhiên (ví dụ: "Tôi muốn đặt lịch xét nghiệm vào tầm 10h, từ ngày 15 đến 20/1"), hệ thống sẽ phân tích, xử lý và phản hồi (ví dụ: "Lịch của bạn đã được đặt vào 10h ngày 15/1. Nhân viên sẽ đến địa chỉ X.").

Ứng dụng sử dụng kiến trúc client-server, với backend xử lý logic AI và database, frontend là giao diện chat đơn giản. Đề tài kết hợp AI cơ bản để nâng cao trải nghiệm người dùng trong lĩnh vực y tế.

2.2. Các tính năng chính

- **Đặt lịch xét nghiệm tại nhà:** Phân tích yêu cầu (loại xét nghiệm, thời gian, địa chỉ) qua AI, lưu lịch vào database và xác nhận.
- **Tư vấn sức khỏe cơ bản:** Phân tích triệu chứng nhập vào (cân nhắc mở rộng text/voice), gợi ý loại xét nghiệm hoặc lời khuyên (sử dụng NLP để phát hiện rủi ro cơ bản).
- **Quản lý lịch sử:** Dashboard xem lịch hẹn cũ, chỉnh sửa hoặc hủy.

- **Chatbot thông minh:** Hỗ trợ fallback (hỏi lại nếu không hiểu), đảm bảo privacy (không lưu dữ liệu nhạy cảm).
- **Tích hợp thông báo:** Gửi email/SMS xác nhận (sử dụng API đơn giản).

2.3. Đối tượng người dùng và lợi ích xã hội

- **Đối tượng:** Người dùng cá nhân (dân văn phòng, người cao tuổi, cư dân đô thị), không nhắm nhóm cụ thể nhưng thiết thực cho mọi lứa tuổi.
- **Lợi ích:** Giảm thời gian di chuyển đến bệnh viện, tăng nhận thức sức khỏe, hỗ trợ phòng ngừa bệnh tật. Trong bối cảnh Việt Nam năm 2026, ứng dụng góp phần vào chuyển đổi số y tế, giảm tải hệ thống y tế công.

3. CÔNG NGHỆ SỬ DỤNG

3.1. Kiến trúc hệ thống

Hệ thống HomeLab được thiết kế theo kiến trúc phân lớp, tách biệt rõ ràng giữa giao diện người dùng, tầng xử lý nghiệp vụ, tầng dữ liệu và tầng tích hợp AI.

- **Frontend (ReactJS):** Chỉ hiển thị giao diện chat-based, tiếp nhận yêu cầu người dùng bằng ngôn ngữ tự nhiên, hiển thị phản hồi theo thời gian thực, ...
- **Backend (NodeJS):** Đóng vai trò máy chủ trung tâm, xử lý nghiệp vụ đặt lịch xét nghiệm, quản lý người dùng, xác thực, đồng thời điều phối luồng hội thoại (trạng thái/phiên chat), gọi dịch vụ AI để phân tích ngôn ngữ tự nhiên và tạo phản hồi.
- **Database (MySQL):** Lưu trữ dữ liệu tài khoản người dùng, lịch hẹn xét nghiệm, phiên chat và lịch sử tin nhắn; phục vụ tra cứu và thống kê.
- **AI Layer (LLM/NLP API):** Thực hiện phân tích ngôn ngữ tự nhiên nhằm nhận diện ý định (intent), trích xuất thực thể (entities) như thời gian, loại xét nghiệm, địa chỉ..., và hỗ trợ sinh phản hồi tư vấn sơ bộ theo nguyên tắc an toàn (không chẩn đoán).

Mô tả sơ đồ kiến trúc:

Client (Browser/React) → (HTTP REST API / WebSocket) → Backend (NodeJS/Express) → (AI API: OpenAI hoặc tương đương)

Backend (NodeJS/Express) ↔ Database (MySQL)

Trong đó, giao tiếp thời gian thực giữa Frontend và Backend được thực hiện qua WebSocket để đảm bảo trải nghiệm chat mượt và phản hồi nhanh.

3.2. Công nghệ backend

Backend được xây dựng bằng NodeJS nhằm đáp ứng tốt các yêu cầu về xử lý API, giao tiếp thời gian thực và tích hợp với các dịch vụ AI bên ngoài.

- **Nền tảng/Framework: NodeJS + ExpressJS**

  ExpressJS được sử dụng để xây dựng các API phục vụ đăng nhập, quản lý lịch hẹn, lịch sử và các thao tác nghiệp vụ liên quan. Kiến trúc backend được tổ chức theo hướng tách lớp (routes/controllers/services) nhằm đảm bảo rõ ràng và dễ mở rộng.

- **Real-time Communication: WebSocket (Socket.IO hoặc ws)**

  Backend duy trì kênh giao tiếp hai chiều với client để gửi/nhận tin nhắn chat theo thời gian thực, đồng thời cập nhật trạng thái hội thoại (ví dụ: thiếu thông tin ngày giờ, thiếu địa chỉ... sẽ hỏi bổ sung).

- **Database: MySQL**

  Dữ liệu được lưu trong MySQL để đảm bảo tính bền vững và nhất quán (tài khoản, lịch hẹn, tin nhắn, phiên chat). Backend truy cập DB thông qua ORM (ví dụ **Prisma** hoặc **Sequelize**) để giảm lỗi truy vấn thủ công và tối ưu quản lý schema.

- **Authentication & Security: JWT (hoặc session-based)**

  Hệ thống áp dụng cơ chế xác thực bằng JWT để bảo vệ API. Các biện pháp cơ bản như kiểm soát truy cập theo token, kiểm tra dữ liệu đầu vào và giới hạn tần suất (rate limiting) được cân nhắc triển khai nhằm tăng độ an toàn khi hệ thống kết nối ra bên ngoài (AI API).

3.3. Công nghệ frontend

Frontend được xây dựng bằng ReactJS với định hướng tối ưu trải nghiệm người dùng theo mô hình chat-based, giảm thao tác form và tăng tính tự nhiên khi tương tác.

- **Framework: ReactJS**

  ReactJS được sử dụng để xây dựng giao diện theo dạng component, phù hợp cho việc tổ chức UI chat và các màn hình chức năng đi kèm (đặt lịch, lịch sử, hồ sơ,...).

- **Routing: React Router**

Dùng để điều hướng giữa các trang: trang chat chính, trang lịch hẹn, trang đăng nhập/đăng ký, trang chi tiết lịch hẹn,...

- **Giao tiếp API: Fetch/Axios**

  Frontend gọi các REST API để thực hiện các chức năng như đăng nhập, lấy danh sách lịch hẹn, cập nhật thông tin người dùng,...

- **Real-time Chat: Socket.IO Client (hoặc WebSocket Client)**

  Frontend kết nối kênh WebSocket tới backend để gửi tin nhắn và nhận phản hồi ngay lập tức, tạo trải nghiệm tương tự các ứng dụng chat phổ biến.

- **Thư viện UI (tùy chọn): Material UI / Ant Design / TailwindCSS**

  Dùng để xây dựng giao diện tối giản, dễ dùng (đặc biệt hướng tới nhóm người dùng ít thao tác công nghệ).

3.4. Tích hợp AI và các công cụ hỗ trợ

Hệ thống tích hợp AI theo mô hình "AI hỗ trợ tác vụ" (task-oriented assistant), tập trung vào việc hiểu yêu cầu bằng tiếng Việt và tự động hóa quy trình đặt lịch, đồng thời cung cấp tư vấn sức khỏe ở mức sơ bộ, đúng phạm vi cho phép.

- **AI NLP & Generative: LLM API (OpenAI hoặc tương đương)**

  AI được sử dụng để:

  1. **Nhận diện ý định (intent):** ví dụ đặt lịch xét nghiệm, thay đổi lịch, hủy lịch, tư vấn sơ bộ,...
  2. **Trích xuất thực thể (entities):** ngày/giờ, loại xét nghiệm, địa chỉ, ghi chú,...
  3. **Sinh phản hồi hội thoại:** hướng dẫn người dùng cung cấp đủ thông tin (slot filling) và xác nhận lại dữ liệu trước khi tạo lịch hẹn.

- **Quản lý luồng hội thoại (AI Flow):**

  Backend duy trì **trạng thái phiên chat** (session/state) để đảm bảo hội thoại có ngữ cảnh. Khi người dùng thiếu thông tin quan trọng (ví dụ chưa có thời gian hoặc địa chỉ), hệ thống sẽ hỏi bổ sung theo từng bước và chỉ tạo lịch hẹn khi dữ liệu đã đầy đủ. Cách triển khai có thể theo hai hướng:

  - **Tự xây dựng flow** trong backend (state machine/slot-filling) để chủ động kiểm soát logic; hoặc
  - **Sử dụng framework hỗ trợ** (ví dụ LangChain) khi cần mở rộng hội thoại thành nhiều bước/prompt phức tạp.

- **Công cụ tích hợp:**

  Backend sử dụng thư viện HTTP client của NodeJS (ví dụ **axios** hoặc **node-fetch**) để gọi AI API, đồng thời triển khai cơ chế timeout/retry hợp lý nhằm ổn định phản hồi. Với kênh realtime, backend gửi phản hồi AI về client qua WebSocket để đảm bảo trải nghiệm chat liền mạch.

3.4.1. Cơ sở tri thức y khoa (Medical Know-how) của HomeLab

Trong phạm vi khóa luận, HomeLab không thực hiện chẩn đoán mà cung cấp tư vấn sức khỏe ở mức cơ bản, tập trung vào: (1) hướng dẫn chuẩn bị trước xét nghiệm, (2) gợi ý nhóm xét nghiệm tham khảo dựa trên triệu chứng phổ biến, và (3) cảnh báo dấu hiệu nguy hiểm cần đi khám sớm. Để đảm bảo nội dung tư vấn có căn cứ và hạn chế "hallucination", hệ thống xây dựng **Knowledge Base (KB)** từ **guideline** và **FAQ** đã được chuẩn hóa.

(1) Nguyên tắc lựa chọn nguồn tri thức

KB sử dụng thông tin từ các nguồn có độ tin cậy cao, nội dung dành cho người dân/ bệnh nhân, có tính hướng dẫn (guideline/leaflet/FAQ), tránh nội dung chuyên sâu mang tính chẩn đoán điều trị. Các nguồn được ưu tiên gồm:

- **Hướng dẫn chuẩn bị xét nghiệm và yêu cầu nhịn ăn** (fasting instructions) từ các tổ chức y tế/đơn vị xét nghiệm uy tín như NHS, MedlinePlus, Mayo Clinic, Cleveland Clinic.

  Ví dụ: nhiều xét nghiệm máu yêu cầu nhịn ăn 8–12 giờ, thường chỉ uống nước, nhưng mức nhịn ăn có thể thay đổi theo loại xét nghiệm.

- **Cảnh báo dấu hiệu nguy hiểm (red flags)** dựa trên hướng dẫn nhận biết tình trạng cần chăm sóc y tế khẩn cấp. Ví dụ: khó thở, đau/tức ngực, lú lẫn, co giật...

(2) Phạm vi tri thức y tế được đưa vào KB

KB được tổ chức theo 3 nhóm nội dung phù hợp với mô tả hệ thống và tính năng "tư vấn cơ bản + gợi ý xét nghiệm" đã nêu trong báo cáo.

- Nhóm A – Hướng dẫn chuẩn bị trước xét nghiệm (Pre-test nhịn ăn, chỉ uống nước, thời điểm lấy mẫu, những lưu ý phổ biến trước khi xét nghiệm.

  Ví dụ: hướng dẫn nhịn ăn thường gặp cho xét nghiệm mỡ máu, đường huyết, xét nghiệm máu khi bác sĩ yêu cầu nhịn ăn qua đêm.

- **Nhóm B – FAQ quy trình dịch vụ xét nghiệm tại nhà:** đặt lịch, đổi/hủy lịch, quy trình lấy mẫu, thời gian phản hồi, quyền riêng tư dữ liệu (mức hệ thống).

- **Nhóm C – Sàng lọc nguy cơ cơ bản & khuyến nghị đi khám:** tập các "red flags" để hệ thống ưu tiên cảnh báo an toàn, tránh tư vấn sai hướng.

  Ví dụ: khó thở, đau ngực kéo dài, chóng mặt nặng/sudden...

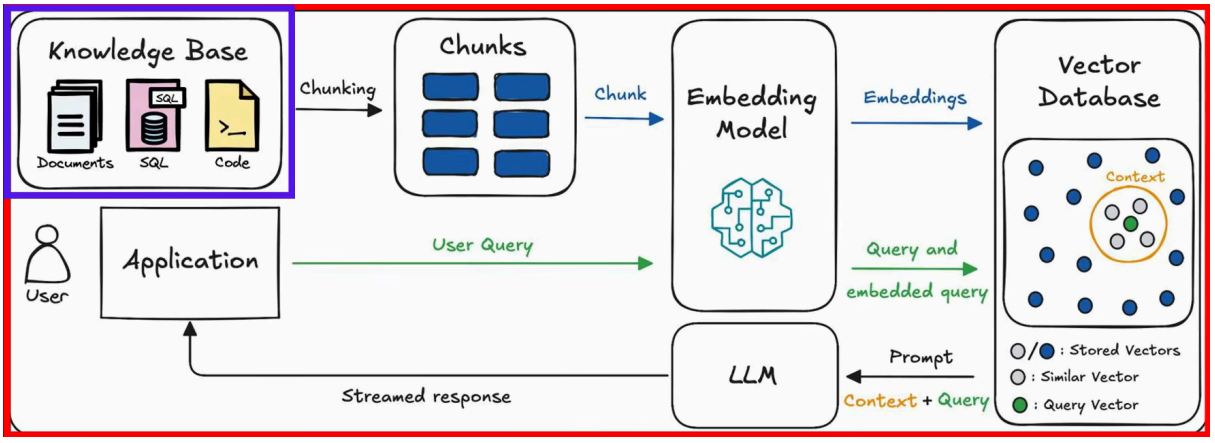
(3) Chuẩn hóa tri thức thành các "mảnh KB" có cấu trúc

Mỗi mục kiến thức được chuẩn hóa thành các bản ghi (knowledge items) gồm: `title`, `content`, `tags`, `test_types`, `risk_level`, `source`, nhằm phục vụ truy xuất và kiểm soát câu trả lời. Điều này giúp tách bạch "nguồn know-how" khỏi mô hình LLM.

→ Đảm bảo nền tảng tri thức cho Homelab

3.4.2. Quy trình RAG (Retrieval-Augmented Generation) cho tư vấn sức khỏe

HomeLab áp dụng RAG để tạo phản hồi tư vấn dựa trên tri thức đã lưu trong KB.



Quy trình gồm 5 bước chính:

Bước 1: Tiền xử lý & chia đoạn (chunking)

Tài liệu guideline/FAQ được chia thành các đoạn ngắn theo đơn vị ý nghĩa (FAQ question/heading). Mỗi chunk tập trung vào một nội dung cụ thể (ví dụ "nhịn ăn trước xét nghiệm", "khi nào cần đi khám ngay") để tránh nhiễu khi truy xuất.

Bước 2: Lập chỉ mục (indexing) bằng embedding + metadata

Mỗi chunk được mã hóa thành vector embedding (sử dụng embedding model sẽ tìm hiểu sau), lưu trong kho tìm kiếm (vector index). Metadata (tags, test_types, risk_level) hỗ trợ lọc theo bối cảnh (đặt lịch/chuẩn bị xét nghiệm/red flags).

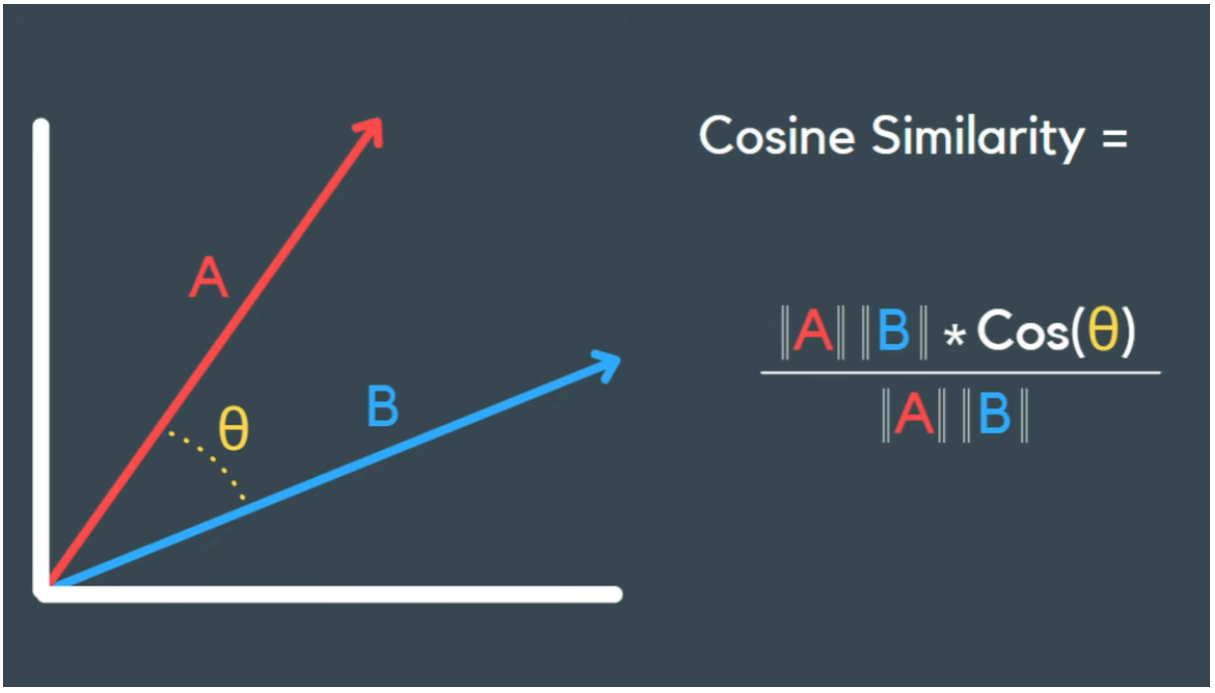
Bước 3: Truy xuất tri thức (retrieval)

Khi người dùng nhập triệu chứng/câu hỏi, backend tạo truy vấn và lấy Top-k chunk liên quan nhất. Hệ thống áp dụng **ngưỡng điểm tương đồng (similarity threshold)**

→ Nếu không có chunk đủ liên quan, hệ thống sẽ chuyển sang chế độ hỏi thêm thông tin hoặc khuyến nghị người dùng gặp bác sĩ (an toàn).

Áp dụng công thức Cosine Similarity: so sánh khoảng cách của vector đa chiều của các chunk sau khi đưa vào embedding model (góc càng bé thì vector càng tương đồng)

→ Embedding vector được sử dụng như là context ngữ cảnh để trả lời câu hỏi của user



Bước 4: Sinh câu trả lời có căn cứ (grounded generation)

LLM nhận vào: (1) câu hỏi người dùng, (2) các chunk đã truy xuất, (3) quy tắc an toàn. LLM được ràng buộc "chỉ trả lời dựa trên KB". Ví dụ, với hướng dẫn nhịn ăn, câu trả lời phải bám theo nội dung guideline phổ biến như nhịn ăn 8–12 giờ (tùy xét nghiệm), thường chỉ uống nước và làm theo chỉ định cụ thể.

Bước 5: Lớp kiểm soát an toàn (safety gate)

Trước khi trả về cho người dùng, hệ thống kiểm tra các trường hợp nhạy cảm:

- Nếu câu hỏi chứa dấu hiệu nguy hiểm, ưu tiên cảnh báo "đi khám ngay" theo KB red flags.

- Không cung cấp chẩn đoán, không kê thuốc, không hướng dẫn điều trị chuyên môn (đúng giới hạn đề tài).

4. CẤU TRÚC BÁO CÁO KHÓA LUẬN

Báo cáo KLTN sẽ khoảng 80-100 trang với các chương sau:

4.1. Chương 1: Giới thiệu

Giới thiệu vấn đề, mục tiêu, phạm vi, ý nghĩa.

4.2. Chương 2: Tổng quan tài liệu và công nghệ

Phân tích các ứng dụng tương tự (Y - HOMEMEDI, Kantesti), tài liệu về AI trong y tế, công nghệ sử dụng.

4.3. Chương 3: Phân tích và thiết kế hệ thống

Yêu cầu chức năng/non-chức năng, use case diagram, class diagram, database schema, thiết kế AI logic.

4.4. Chương 4: Triển khai và thử nghiệm

Mô tả code chi tiết (snippet), cách tích hợp, test cases (unit, integration), kết quả thử nghiệm (performance, accuracy AI).

4.5. Chương 5: Kết quả, đánh giá và kết luận

Kết quả demo, đánh giá ưu nhược điểm, đề xuất cải tiến, kết luận.

4.6. Phụ lục và tài liệu tham khảo

Source code snippet, questionnaire đánh giá, references (APA style, từ Google Scholar, tài liệu Laravel/Hugging Face).